LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 5 TENATANG SINGLE LINKED LIST

Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR……ii

DAFTAR ISI……..iii

BAB 1 PENDAHULUAN……..……1

1.1 Pengertian Praktikum……1

1.2 Tujuan Praktikum……..1

1.3 Persyaratan Praktikum…….……..1

1.4 Tempat dan Waktu Praktikum…….……..2

BAB 2 PEMBAHASAN PRAKTIKUM…….3

2.1 Pengertian SLL…….3

2.2 Langkah-Langkah Praktikum…….3

BAB 3 PENUTUP……..……15

3.1 Kesimpulan……...……15

LAMPIRAN……...……16

BAB 1
PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.

2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.

3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.

4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.

5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian SLL

1. Definisi Umum

Single Linked List (SLL) adalah salah satu jenis struktur data linear yang terdiri dari sekumpulan elemen yang disebut **node**, di mana setiap node terhubung satu sama lain secara berurutan melalui satu buah *pointer* (penunjuk).

Berikut adalah poin-poin utama untuk memahami SLL:

- **Struktur Node:** Setiap node dalam SLL terdiri dari dua bagian utama, yaitu **data** (informasi yang disimpan) dan **next** (sebuah *pointer* yang menyimpan alamat memori dari node berikutnya).
- **Urutan Linear:** SLL bersifat linear karena setiap node hanya menunjuk ke satu node setelahnya, sehingga membentuk rantai data. Node terakhir dalam list biasanya menunjuk ke null, yang menandakan akhir dari *linked list*.
- **Alokasi Dinamis:** Berbeda dengan *array* yang ukurannya harus ditentukan di awal, SLL menggunakan alokasi memori dinamis. Ini berarti kita dapat menambah atau menghapus node kapan saja saat program berjalan, sesuai dengan kebutuhan memori.
- **Akses Data:** Karena SLL adalah struktur data *sequential*, kita tidak bisa langsung mengakses node di posisi tertentu (seperti list[5] pada array). Kita harus melakukan *traversal* (penelusuran) dari node pertama (*head*) satu per satu hingga mencapai posisi yang diinginkan.

Secara sederhana, Anda bisa membayangkan SLL seperti rantai kereta api, di mana setiap gerbong (node) menyimpan muatan (data) dan memiliki pengait (pointer) yang terhubung ke gerbong berikutnya. Jika salah satu pengait dilepas atau diubah, urutan atau struktur rantai tersebut akan berubah secara otomatis.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 4 buah class yang pertama Adalah **NodeSLL_2511533023:**

```
public class NodeSLL_2511533023 {
        // node bagian data
        int data_3023;
        // pointer ke node berikutnya
        NodeSLL_2511533023 next_3023;
        // Konstruktor menginisialisasi node dengan data

        public NodeSLL_2511533023(int data_3023) {
                this.data_3023 = data_3023;
                this.next_3023 = null;
        }

}
```

Hasil Output:
Tidak ada karena ini merupakan class blueprint atau kerangka saja yang nanti hasil nya dapat di panggil pakai class driver.

Class ke-2 adalah TambahSLL_2511533023:

```
public class TambahSLL_2511533023 {
   public static NodeSLL_2511533023 insertAtFront(NodeSLL_2511533023 head, int value) {
      NodeSLL_2511533023 new_node = new NodeSLL_2511533023(value);
      new_node.next_3023 = head;
      return new_node;
   }

   // fungsi menambahkan node di akhir SLL
   public static NodeSLL_2511533023 insertAtEnd(NodeSLL_2511533023 head, int value) {
      // buat sebuah node dengan sebuah nilai
      NodeSLL_2511533023 newNode = new NodeSLL_2511533023(value);
      // jika list kosong maka node jadi head
      if (head == null) {
         return newNode;
      }
      // simpan head ke variabel sementara
      NodeSLL_2511533023 last = head;
      // telusuri ke node akhir
      while (last.next_3023 != null) {
         last = last.next_3023;
      }
```

```
        // ubah pointer
        last.next_3023 = newNode;
        return head;
    }

    static NodeSLL_2511533023 GetNode(int data) {
        return new NodeSLL_2511533023(data);
    }

    static NodeSLL_2511533023 insertPos(NodeSLL_2511533023 headNode, int
position, int value) {
        NodeSLL_2511533023 head = headNode;
        if (position < 1)
            System.out.print("Invalid position");
        if (position == 1) {
            NodeSLL_2511533023 new_node = new NodeSLL_2511533023(value);
            new_node.next_3023 = head;
            return new_node;
        } else {
            while (position-- != 0) {
                if (position == 1) {
                    NodeSLL_2511533023 newNode = GetNode(value);
                    newNode.next_3023 = headNode.next_3023;
                    headNode.next_3023 = newNode;
                    break;
                }
                headNode = headNode.next_3023;
            }
            if (position != 1)
                System.out.print("Posisi di luar jangkauan");
        }
        return head;
    }

    public static void printList(NodeSLL_2511533023 head) {
        NodeSLL_2511533023 curr = head;
        while (curr.next_3023 != null) {
            System.out.print(curr.data_3023 + "-->");
            curr = curr.next_3023;
        }
        if (curr.next_3023 == null) {
            System.out.print(curr.data_3023);
        }
        System.out.println();
    }
```

```
    public static void main(String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511533023 head = new NodeSLL_2511533023(2);
        head.next_3023 = new NodeSLL_2511533023(3);
        head.next_3023.next_3023 = new NodeSLL_2511533023(5);
        head.next_3023.next_3023.next_3023 = new NodeSLL_2511533023(6);

        // cetak list asli
        System.out.print("Senarai berantai awal:");
        printList(head);

        // tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data = 1;
        head = insertAtFront(head, data);

        // cetak update list
        printList(head);

        // tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2 = 7;
        head = insertAtEnd(head, data2);

        // cetak update list
        printList(head);

        System.out.print("tambah 1 simpul ke data 4: ");
        int data3 = 4;
        int pos = 4;
        head = insertPos(head, pos, data3);

        // cetak update list
        printList(head);
    }
}
}
```

Hasil Output:

Senarai berantai awal:2-->3-->5-->6

tambah 1 simpul di depan: 1-->2-->3-->5-->6

tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7

tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

Class ke-3 PencarianSLL_2511533023:

```
public class PencarianSLL_2511533023 {
    static boolean searchKey_3023(NodeSLL_2511533023 head_3023, int key_3023) {
        NodeSLL_2511533023 curr_3023 = head_3023;
        while (curr_3023 != null) {
            if (curr_3023.data_3023 == key_3023)
                return true;
            curr_3023 = curr_3023.next_3023;
        }
        return false;
    }

    public static void traversal_3023(NodeSLL_2511533023 head_3023) {
        // mulai dari head
        NodeSLL_2511533023 curr_3023 = head_3023;
        // telusuri sampai pointer null
        while (curr_3023 != null) {
            System.out.print(" " + curr_3023.data_3023);
            curr_3023 = curr_3023.next_3023;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511533023 head_3023 = new NodeSLL_2511533023(14);
        head_3023.next_3023 = new NodeSLL_2511533023(21);
        head_3023.next_3023.next_3023 = new NodeSLL_2511533023(13);
        head_3023.next_3023.next_3023.next_3023 = new NodeSLL_2511533023(30);
        head_3023.next_3023.next_3023.next_3023.next_3023 = new
NodeSLL_2511533023(10);

        System.out.print("Penelusuran SLL : ");
        traversal_3023(head_3023);

        // data yang akan dicari
        int key_3023 = 30;
        System.out.print("cari data " + key_3023 + " = ");
        if (searchKey_3023(head_3023, key_3023))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

Hasil Output:

Penelusuran SLL : 14 21 13 30 10

cari data 30 = ketemu

Class ke-4 HapusSLL_2511533023:

```
public class HapusSLL_2511533023 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511533023 deleteHead_3023(NodeSLL_2511533023 head_3023) {
        // jika SLL kosong
        if (head_3023 == null)
            return null;
        // pindahkan head ke node berikutnya
        head_3023 = head_3023.next_3023;
        // Return head baru
        return head_3023;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533023 removeLastNode_3023(NodeSLL_2511533023 head_3023) {
        // jika list kosong, return null
        if (head_3023 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head_3023.next_3023 == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511533023 secondLast_3023 = head_3023;
        while (secondLast_3023.next_3023.next_3023 != null) {
            secondLast_3023 = secondLast_3023.next_3023;
        }
        // hapus node terakhir
        secondLast_3023.next_3023 = null;
        return head_3023;
    }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533023 deleteNode_3023(NodeSLL_2511533023 head_3023, int position_3023) {
        NodeSLL_2511533023 temp_3023 = head_3023;
        NodeSLL_2511533023 prev_3023 = null;
        // jika linked list null
        if (temp_3023 == null)
            return head_3023;
        // kasus 1: head dihapus
        if (position_3023 == 1) {
```

```
            head_3023 = temp_3023.next_3023;
            return head_3023;
        }

        for (int i_3023 = 1; temp_3023 != null && i_3023 < position_3023; i_3023++) {
            prev_3023 = temp_3023;
            temp_3023 = temp_3023.next_3023;
        }
        // jika ditemukan, hapus node
        if (temp_3023 != null) {
            prev_3023.next_3023 = temp_3023.next_3023;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_3023;
    }

    // fungsi mencetak SLL
    public static void printList_3023(NodeSLL_2511533023 head_3023) {
        NodeSLL_2511533023 curr_3023 = head_3023;
        while (curr_3023.next_3023 != null) {
            System.out.print(curr_3023.data_3023 + "-->");
            curr_3023 = curr_3023.next_3023;
        }
        if (curr_3023.next_3023 == null) {
            System.out.print(curr_3023.data_3023);
        }
        System.out.println();
    }

    // kelas main
    public static void main(String[] args) {
        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511533023 head_3023 = new NodeSLL_2511533023(1);
        head_3023.next_3023 = new NodeSLL_2511533023(2);
        head_3023.next_3023.next_3023 = new NodeSLL_2511533023(3);
        head_3023.next_3023.next_3023.next_3023 = new NodeSLL_2511533023(4);
        head_3023.next_3023.next_3023.next_3023.next_3023 = new
NodeSLL_2511533023(5);
        head_3023.next_3023.next_3023.next_3023.next_3023.next_3023 = new
NodeSLL_2511533023(6);

        // cetak list awal
        System.out.println("list awal: ");
        printList_3023(head_3023);
```

```
        // hapus head
        head_3023 = deleteHead_3023(head_3023);
        System.out.println("List setelah head dihapus: ");
        printList_3023(head_3023);

        // hapus node terakhir
        head_3023 = removeLastNode_3023(head_3023);
        System.out.println("List setelah simpul terakhir di hapus: ");
        printList_3023(head_3023);

        // Deleting node at position 2
        int position_3023 = 2;
        head_3023 = deleteNode_3023(head_3023, position_3023);

        // Print list after deletion
        System.out.println("List setelah posisi 2 dihapus: ");
        printList_3023(head_3023);
    }
}
```

Hasil Output:
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

Tadi sudah saya katakan bahwa NodeSLL berperan sebagai blue print dan class tamabah , pencarian dan hapus sebagai drivernya(isinya).

BAB 3
PENUTUP

3.1 Kesimpulan

Dari praktikum ini saya menjadi lebih memahami bagaimana cara sistem mengelola memori secara dinamis melalui manipulasi pointer daripada sekadar menggunakan array yang kaku. Saya belajar bahwa kunci utama dalam operasi seperti menambah, mencari, hingga menghapus node terletak pada ketelitian saya dalam mengatur referensi alamat antar-node agar alurnya tidak terputus, yang meskipun cukup menantang karena mengharuskan saya melakukan traversal dari head untuk operasi tertentu, pada akhirnya memberikan saya pemahaman yang jauh lebih dalam tentang bagaimana efisiensi algoritma bekerja di balik layar dalam menyusun struktur data yang kompleks.

Tugas SLL:
Class pertama Pasien_2511533023:

```
public class Pasien_2511533023 {
   private String namaPasien_3023; // Nama Pasien
   private String penyakit_3023; // Keluhan / Penyakit
   private int nomorAntrian_3023; // Nomor Antrian
   private Pasien_2511533023 next_3023; // Pointer next ke node Pasien berikutnya

   // Constructor untuk menginisialisasi semua atribut
   public Pasien_2511533023(String nama_3023, String keluhan_3023, int
noAntrian_3023) {
      this.namaPasien_3023 = nama_3023;
      this.penyakit_3023 = keluhan_3023;
      this.nomorAntrian_3023 = noAntrian_3023;
      this.next_3023 = null;
   }

   // Getter untuk setiap atribut
   public String getNamaPasien_3023() { return namaPasien_3023; }
   public String getPenyakit_3023() { return penyakit_3023; }
   public int getNomorAntrian_3023() { return nomorAntrian_3023; }
   public Pasien_2511533023 getNext_3023() { return next_3023; }

   // Setter untuk setiap atribut termasuk pointer next
   public void setNext_3023(Pasien_2511533023 next_3023) { this.next_3023 =
next_3023; }
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dikendalikan oleh class RumahSakit_2511533023 sebagai driver.

Class kedua RumahSakit_2511533023:

```
import java.util.Scanner;

public class RumahSakit_2511533023 {
   private Pasien_2511533023 head_3023 = null; // Pointer head ke antrian pasien
   private int counter_3023 = 1; // Counter nomor antrian otomatis

   // Method untuk mendaftarkan pasien (Insert at Tail)
   public void daftarkanPasien_3023(String nama_3023, String keluhan_3023) {
      Pasien_2511533023 baru_3023 = new Pasien_2511533023(nama_3023,
keluhan_3023, counter_3023++);
      if (head_3023 == null) {
```

```
            head_3023 = baru_3023; // Jika list kosong, jadi head
        } else {
            Pasien_2511533023 temp_3023 = head_3023;
            while (temp_3023.getNext_3023() != null) { // Cari node terakhir
                temp_3023 = temp_3023.getNext_3023();
            }
            temp_3023.setNext_3023(baru_3023); // Hubungkan node terakhir ke node
baru
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
baru_3023.getNomorAntrian_3023());
    }

    // Method untuk memanggil pasien (Delete Head)
    public void panggilPasien_3023() {
        if (head_3023 == null) {
            System.out.println("Antrian kosong!"); // Cek list kosong
        } else {
            System.out.println("Memanggil pasien: " +
head_3023.getNamaPasien_3023());
            head_3023 = head_3023.getNext_3023(); // Geser head ke berikutnya
        }
    }

    // Method untuk menampilkan antrian (Display)
    public void tampilkanAntrian_3023() {
        if (head_3023 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        Pasien_2511533023 temp_3023 = head_3023;
        while (temp_3023 != null) { // Iterasi dari head hingga null
            System.out.println("[" + temp_3023.getNomorAntrian_3023() + "] " +
temp_3023.getNamaPasien_3023() + " - " + temp_3023.getPenyakit_3023());
            temp_3023 = temp_3023.getNext_3023();
        }
    }

    // Method untuk mencari pasien (Search)
    public void cariPasien_3023(String nama_3023) {
        Pasien_2511533023 temp_3023 = head_3023;
        boolean ketemu_3023 = false;
        while (temp_3023 != null) { // Cari berdasarkan nama (case-insensitive)
            if (temp_3023.getNamaPasien_3023().equalsIgnoreCase(nama_3023)) {
```

```
                System.out.println("Pasien ditemukan: " +
temp_3023.getNamaPasien_3023() + ", Keluhan: " +
temp_3023.getPenyakit_3023());
                ketemu_3023 = true;
                break;
            }
            temp_3023 = temp_3023.getNext_3023();
        }
        if (!ketemu_3023) System.out.println("Pasien tidak ditemukan.");
    }

    // Method untuk cek status antrian
    public void cekStatus_3023() {
        if (head_3023 == null) {
            System.out.println("Antrian kosong.");
        } else {
            int total_3023 = 0;
            Pasien_2511533023 temp_3023 = head_3023;
            while (temp_3023 != null) { total_3023++; temp_3023 =
temp_3023.getNext_3023(); } // Hitung jumlah node
            System.out.println("Total pasien: " + total_3023 + ". Pasien terdepan: " +
head_3023.getNamaPasien_3023());
        }
    }

    // Method main untuk menjalankan menu program
    public static void main(String[] args) {
        RumahSakit_2511533023 rs_3023 = new RumahSakit_2511533023();
        Scanner sc_3023 = new Scanner(System.in);
        int pilihan_3023 = 0;

        while (pilihan_3023 != 6) { // Looping menu
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511533023 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3023 = sc_3023.nextInt();
            sc_3023.nextLine(); // Clear buffer

            switch (pilihan_3023) {
                case 1:
                    System.out.print("Masukkan Nama Pasien: ");
```

```
                String nama_3023 = sc_3023.nextLine();
                System.out.print("Masukkan Keluhan : ");
                String keluhan_3023 = sc_3023.nextLine();
                rs_3023.daftarkanPasien_3023(nama_3023, keluhan_3023);
                break;
            case 2: rs_3023.panggilPasien_3023(); break;
            case 3: rs_3023.tampilkanAntrian_3023(); break;
            case 4:
                System.out.print("Masukkan nama yang dicari: ");
                rs_3023.cariPasien_3023(sc_3023.nextLine()); break;
            case 5: rs_3023.cekStatus_3023(); break;
            case 6: System.out.println("Keluar."); break;
            default: System.out.println("Pilihan tidak valid!");
        }
    }
    sc_3023.close();
  }
}
```

Hasil Output:

```
=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: jack
Masukkan Keluhan : batuk
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: yanto
Masukkan Keluhan : demam
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
```

3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
[1] jack - batuk
[2] yanto - demam

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari: yanto
Pasien ditemukan: yanto, Keluhan: demam

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Total pasien: 2. Pasien terdepan: jack

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil pasien: jack

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 5
Total pasien: 1. Pasien terdepan: yanto

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari: jack
Pasien tidak ditemukan.

=== Antrian Rumah Sakit NIM: 2511533023 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Keluar.
Fungsi dari program ini Adalah memanggil atribut yang sudah kita bikin di class Pasien_2511533023 yang di mana itu merupakan kerangka /blueprint untuk program ini.